数据结构

第 3 章 栈和队列

本章知识要点：

- 栈
- 栈的应用举例
- 队列
- 队列的应用举例

学习目标

1. 掌握栈和队列的特点，并能在相应的应用问题中正确选用
2. 熟练掌握栈的两种存储结构的基本操作实现算法，特别应注意栈满和栈空的条件
3. 熟练掌握循环队列和链队列的基本操作实现算法，特别注意队满和队空的条件

3.1 栈 stack

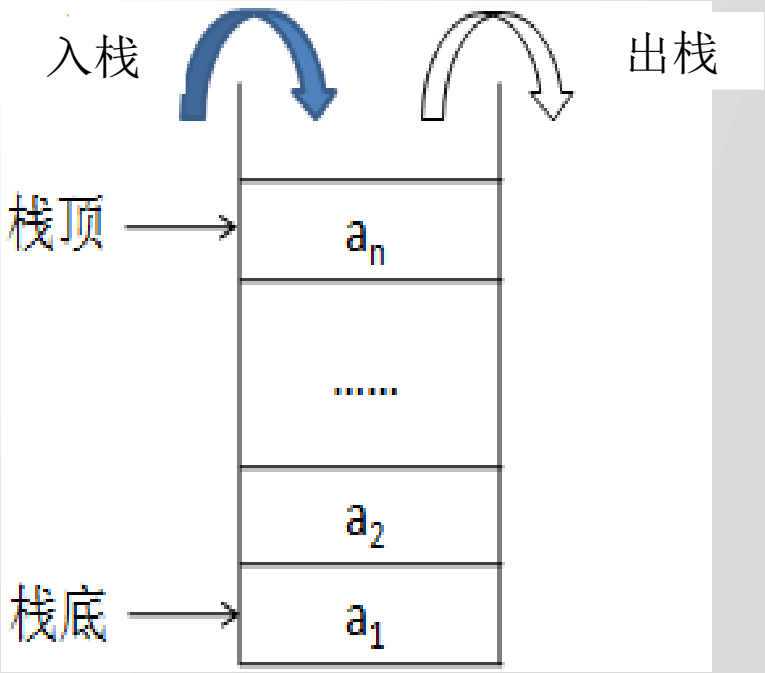
- 栈的定义及基本运算
- 栈的存储实现和运算实现

3.1.1 栈的定义和基本运算

1. 定义： 限制在表的一端（栈顶）进行插入和删除操作的线性表

2. 逻辑结构:与线性表相同，仍为一对一关系

3. 存储结构： 用顺序栈或链栈存储均可



4. 运算规则

只能在栈顶运算，且访问结点时依照后进先出（LIFO）或先进后出（FILO）的原则

5. 实现方式

关键是编写入栈和出栈函数，具体实现依顺序栈或链栈的不同而不同

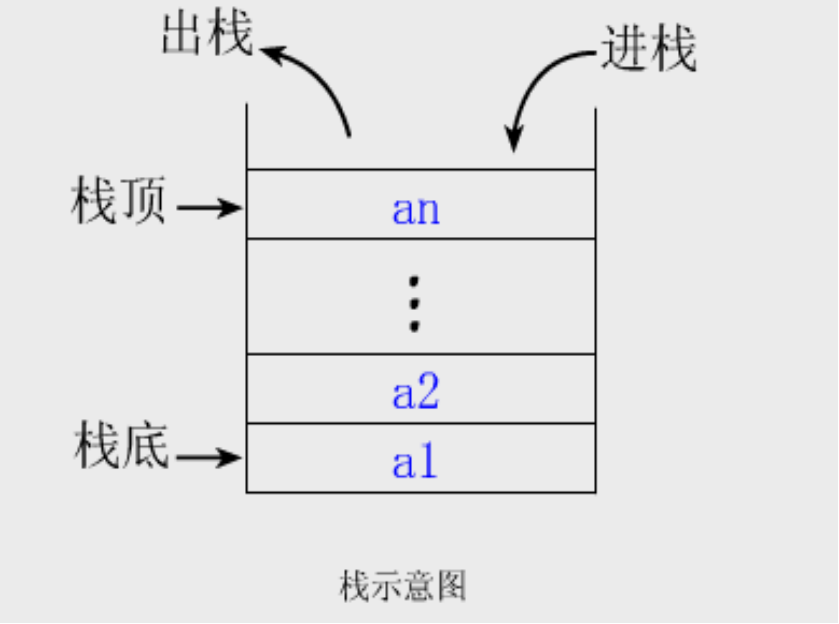
基本操作有建栈、判断栈满、栈空、入栈、出栈、读栈顶元素值等

入栈

- 又称为“压栈”，该操作是将一个数据添加到栈中
- 在入栈操作中，每将一个元素压入栈顶，栈的大小就加 1

出栈

- 每个元素逐个弹出，直到栈变为空



“进” = 压入 =PUSH ( )
“出” = 弹出 =POP( )

栈的基本操作有：

(1)栈初始化： Init_Stack(s)

操作结果是构造了一个空栈。

(2)判栈空： Empty_Stack(s)

操作结果是若 s 为空栈返回为 1 ， 否则返回为 0 。

(3)入栈： Push_Stack(s ， x)

操作结果是在栈 s 的顶部插入一个新元素 x ， x 成为新的栈顶元素。

(4)出栈： Pop_Stack(s)

在栈 s 存在且非空的情况下，操作结果是将栈 s 的顶部元素从栈中删除。

(5)读栈顶元素： Top_Stack(s)

在栈 s 存在且非空情况下，操作结果是读栈顶元素，栈不变化。

栈与一般线性表的对比

栈是一种特殊的线性表，它只能在表的一端（栈顶）进行插入和删除运算

栈与一般线性表的区别：仅在于运算规则不同

一般线性表

逻辑结构：一对一

存储结构：顺序表、链表

运算规则：随机、顺序存取

栈

逻辑结构：一对一

存储结构：顺序栈、链栈

运算规则：后进先出

3.1.2 栈的存储实现和运算实现

1 顺序栈（多用）

2 链栈

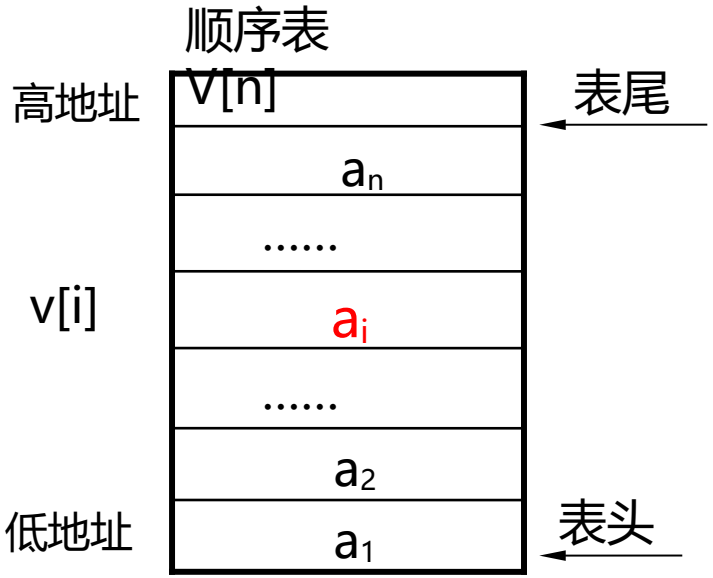
顺序栈

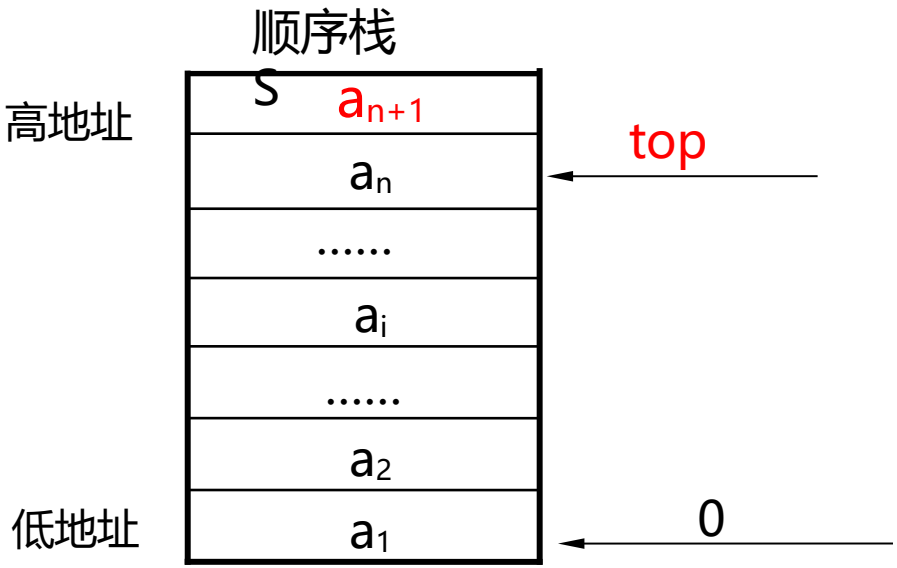
- 顺序栈：以顺序存储方式来存储的栈称为“顺序栈”，即用一片连续的存储空间来存贮栈内元素的值。
- 用顺序表（数组）定义，将栈底位置设置在数组开头，即 0 下标端设为栈底；用 top 来指示栈当前栈顶位置，即栈顶元素的下标。
- 顺序栈的类模板：

```
Class seqlist
 {datatype  data[MAXSIZE];
  int  top;
 }SeqStack ;
```

定义一个指向顺序栈的指针：　　SeqStack *s;

顺序栈与顺序表





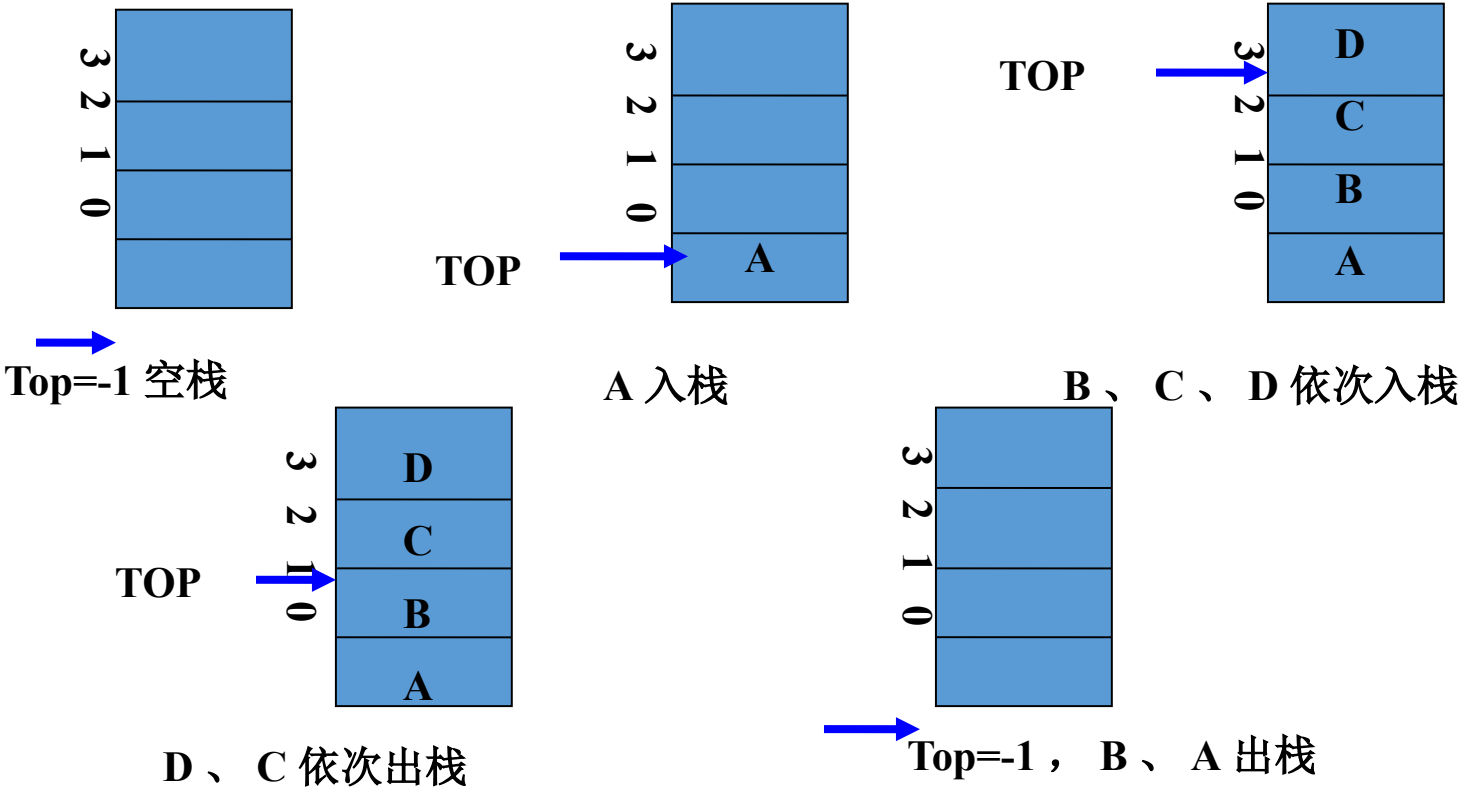
写入： v[i]= a_i

读出： x= v[i]

压入： PUSH (a_{n+1})

弹出： POP (x)

顺序栈操作的示意图


3 2 1 0
Top=-1 空栈
TOP
A
A 入栈
TOP
D
C
B
A
B 、 C 、 D 依次入栈
TOP
D
C
B
A
D 、 C 依次出栈
Top=-1 ， B 、 A 出栈

顺序栈的主要运算

- 特殊情况表示：
  - 栈空： top == -1
  - 栈满： top == maxsize-1
- 运算实现：
  - 入栈：若栈不满 s->top++ ；
  - 出栈：若栈不空 s->top-- ；

(1) 置空栈

【算法 3-1 】栈的初始化算法

```
SeqStack *Init_SeqStack(){
  SeqStack  *s;
  s=new SeqStack;    // 申请栈空间
  if (!s)
    {return NULL;    // 未申请到足够大的存储空间，返回空指针        }
  else
    { s->top=-1;     // 初始化栈顶指针
      return s;      // 申请到栈空间，返回栈空间地址
    }
}
```

(2)判空栈

【算法 3-2 】栈空判别算法

```
int Empty_SeqStack(SeqStack *s){
  if (s->top==-1)
    return 1;    // 栈顶指针指向栈底，空栈
  else
    return 0;
}
```

(3)入栈

(1) 判断是否栈满，若满则出错

(2) 元素 e 压入栈顶

(3) 栈顶指针加 1

【算法 3-3 】入栈算法

```
int Push_SeqStack(SeqStack *s,DataType  x){
  if (s->top==MAXSIZE-1)
    return 0; // 栈满不能入栈，返回错误代码 0
  else
    { s->top++; // 栈顶指针向上移动
      s->data[s->top]=x; // 将 x 至入新的栈顶
      return 1;    // 入栈成功，返回成功代码 1
    }
}
```

(4)出栈

(1) 判断是否栈空，若空则出错

(2) 获取栈顶元素 x

(3) 栈顶指针减 1

【算法 3-4 】出栈算法

```
int Pop_SeqStack(SeqStack *s,DataType  x){
  if (Empty_SeqStack(s))
    return 0;   // 栈空不能出栈，返回错误代码 0
  else
    { x=s->data[s->top];   // 保存栈顶元素值
      s->top--;    // 栈顶指针向下移动
      return 1;        // 返回成功代码 1
    }
}
```

(5)取栈顶元素

(1) 判断是否空栈，若空则返回错误

(2) 否则通过栈顶指针获取栈顶元素

【算法 3-5 】取栈顶元素算法

```
DataType Top_SeqStack(SeqStack *s){
  if (Empty_SeqStack(s))
    return 0;  // 栈空没有栈元素，返回错误代码 0
  else
    return s->data[s->top];  // 返回栈顶元素值
}
```

几点说明：

1. 对于顺序栈，入栈时，首先判栈是否满了，栈满时，不能入栈；否则出现空间溢出，引起错误，这种现象称为上溢。

栈满的条件为： s->top= =MAXSIZE-1

2. 出栈和读栈顶元素操作，先判栈是否为空，为空时不能操作，否则产生错误。通常栈空时常作为一种控制转移的条件。

栈空的条件为： s->top==-1

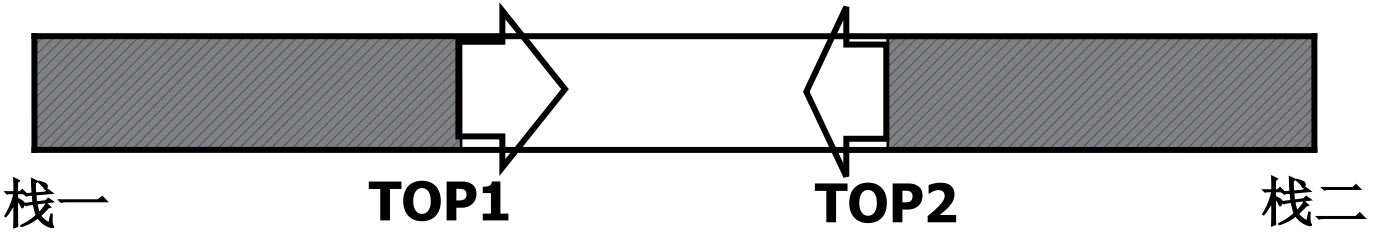
练习

1. 如果一个栈的输入序列为 123456， 能否得到 435612 和 135426 的出栈序列?

435612 中到了 12 顺序不能实现；
135426 可以实现。

❑栈溢出及解决方法

- 栈溢出
  - ➢上溢：栈满时再做进栈运算产生溢出（出错）；
  - ➢下溢：栈空时再做退栈运算产生溢出；
- <u>栈上溢的解决方法</u>
  - ➢方法一：将栈的容量加到足够大，浪费空间。
  - ➢方法二：使用两个（或多个）栈共享存储空间。



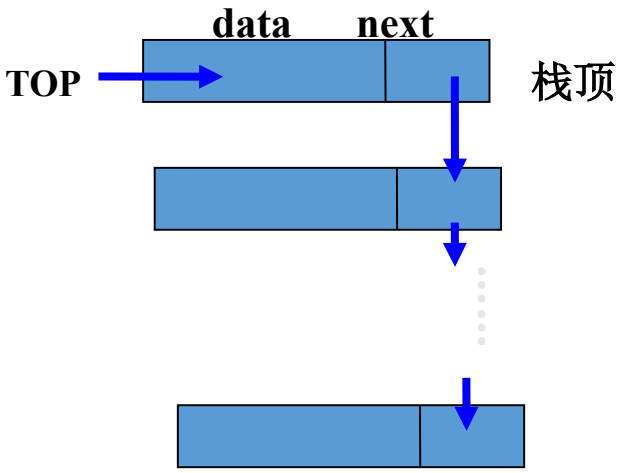
2.链栈

•栈的链式存储结构（当顺序栈的最大容量事先无法估计时，可采用链栈结构）

•通常链栈用单链表表示，因此其结点结构与单链表的结构相同：

```
typedef  struct node
  { datatype data;
    struct node *next;
}StackNode ,   * LinkStack;
```

说明 top 为栈顶指针：　　LinkStack top ；



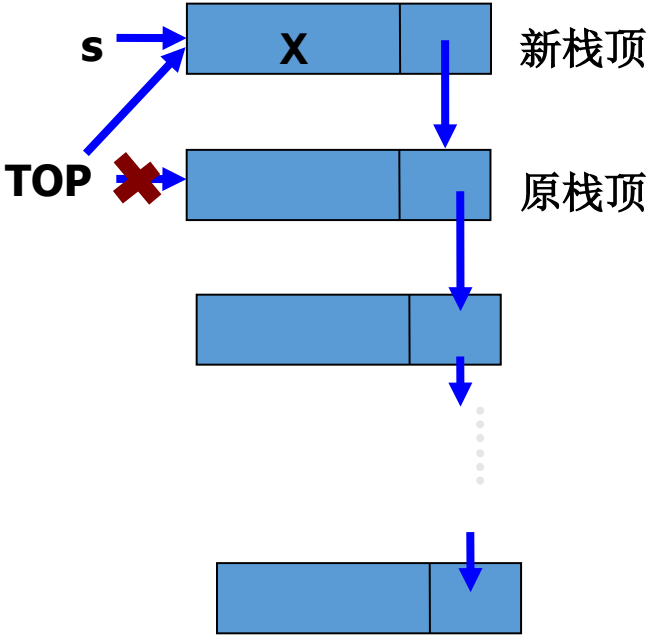
链栈的特点

- 插入和删除（进栈／退栈）仅能在表头位置上（栈顶）进行。
- 链栈中的结点是动态产生的，可不考虑上溢问题。
- 不需附加头结点，栈顶指针就是链表（即链栈）的头指针。
- 始终用表头指向栈顶的元素。
  - 入栈：头插法；
  - 出栈： top = top->next;

若表头为栈底，则入栈和出栈将如何实现？

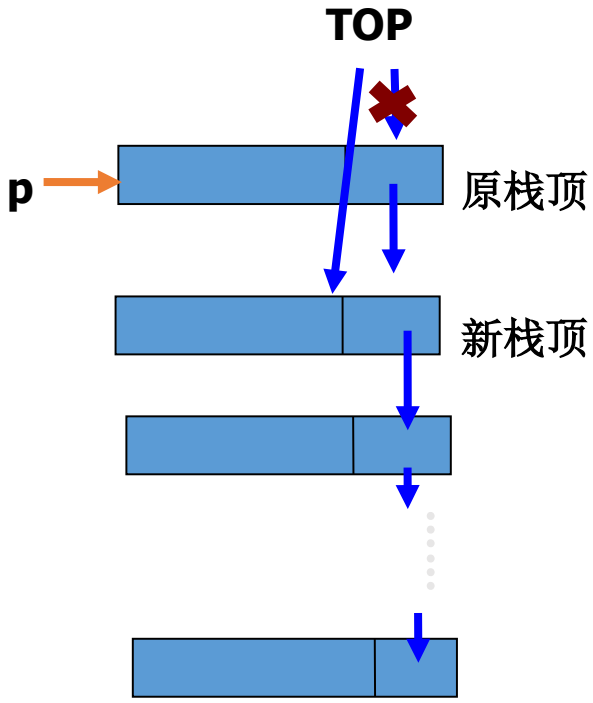
链栈入栈运算

```
LinkStack Push_LinkStack(LinkStack top,DataType x){
  StackNode  *s;
  s=new StackNode;    // 申请新的栈顶结点空间
  s->data=x;    // 将元素值置入结点数据域
  s->next=top;    // 原栈顶结点作为新结点后继
  top=s;                // 将新结点置为栈顶
  return top;
}
```



链栈出栈运算

```
LinkStack Pop_LinkStack(LinkStack top,DataType *x){
  StackNode *p;
  if (top==NULL)
    return NULL;    // 栈空不能出栈，返回空指针
  else
   { *x=top->data;    // 保存栈顶元素值
     p=top;
     top=top->next;    // 置新的栈顶指针
     delete p;         // 释放原栈顶元素结点空间
     return  top;      // 出栈成功，返回新的栈顶指针
   }
}
```



3.2 栈的应用举例

由于栈的“后进先出”特点，在很多实际问题中都利用栈做一个辅助的数据结构来进行求解。

例如：数制转换（十转 N）

用栈暂存低位值

例 3.1 将十进制数 N 转换为 r 进制的数，其转换方法利用辗转相除法。

以 N=3467， r=8 为例转换方法如下：

计算顺序

N	N / 8 （整除）	N % 8 （求余）	
3467	433	3	低
433	54	1	
54	6	6	
6	0	6	高

输出顺序

所以：$(3467)_{10} = (6613)_{8}$

算法思想：当 N>0 时重复 1， 2

1. 若 N≠0，则将 N % r 压入栈 s 中，执行 2;
   若 N=0，将栈 s 的内容依次出栈， 算法结束。
2. 用 N / r 代替 N

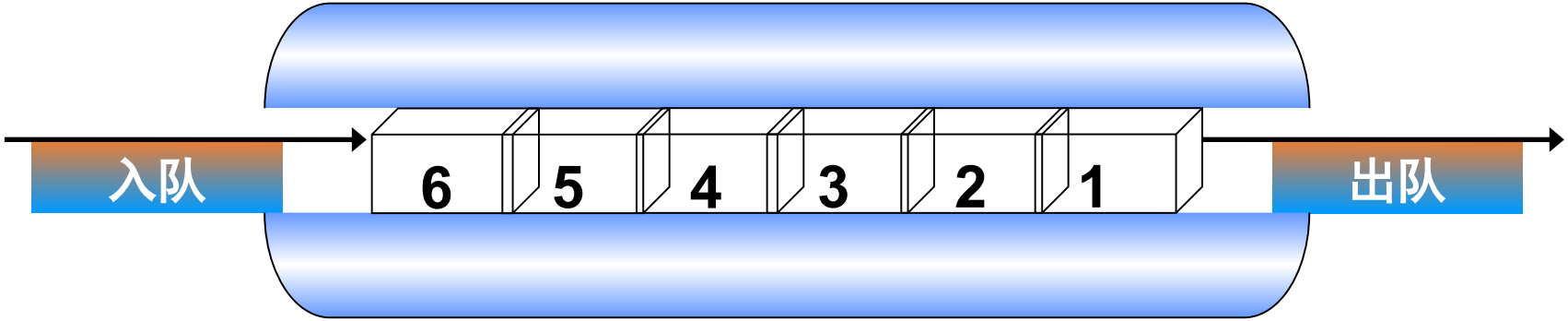
【算法 3-10 】数制转换算法一

```
typedef  int DataType;
void conversion(int N,int r){
  SeqStack *s; // 定义一个顺序栈
  DateType x;
  Init_SeqStack(s);
  while (N)
   { Push_SeqStack(s,N%r);  // 余数入栈
     N=N/r;                 // 商作为被除数继续
   }
```

```
while (!Empty_SeqStack(s))  // 余数按顺序出栈
   { pop_SeqStack(s ,   x);
     cout<<x ;   }
  cout<<endl;
}
```

3.3.1 队列的定义及基本运算

- 在实际问题中经常使用一种“先进先出” (FIFO---First In First Out) 的数据结构：即插入在表一端进行，删除在表的另一端进行，这种数据结构称为队或队列。



从队列头移除元素

在队列尾加入元素

队列的基本概念

- 队头 (front)： 允许删除的一端
- 队尾 (rear)： 允许插入的一端
- 空队列：队列中没有元素
- 入队：队列的插入运算，即插入新的队尾元素。
- 出队：队列的删除运算，即删除队首元素。
- 运算原则：先进先出

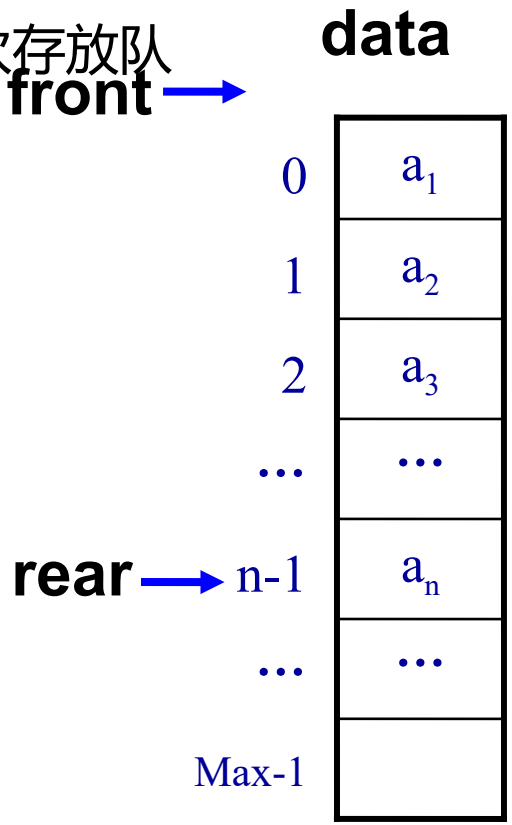
队列的基本运算

- 队列的初始化
- 入队，在队列的队尾插入一个元素 x
- 出队，在队列非空的情况下删除队首元素
- 取队头元素，在队列非空的情况下读取队首元素的值
- 判队列是否为空

顺序队列

顺序队列的定义：

队列的顺序存储结构，用一组连续的存储单元依次存放队列中的元素。



顺序队列的类型说明：

```
typedef struct {
        datatype  data[maxsize];
        int front,rear;    /* 队首、队尾 */
} Sequeue;
```

队首位置： f 总是指向当前队头元素的前一个位置。

队尾位置： r 指向当前队尾元素的位置。

- 申请一个顺序队的存储空间：

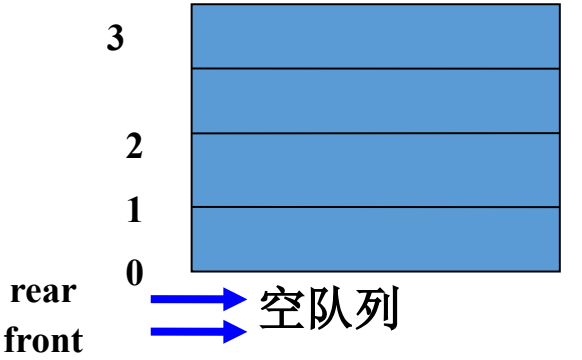
  sq=new Sequeue;

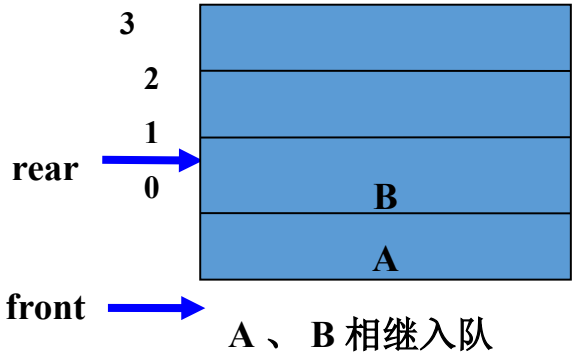
- 队列的数据区为：

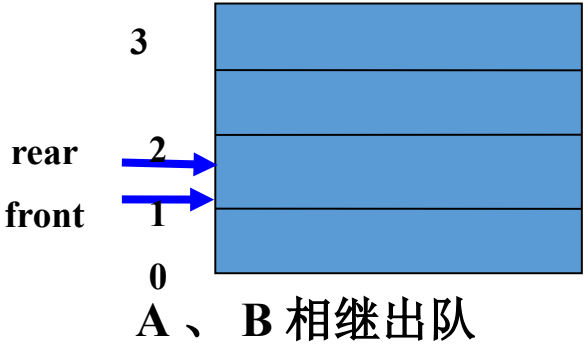
  sq->data[0]---sq->data[MAXSIZE -1]

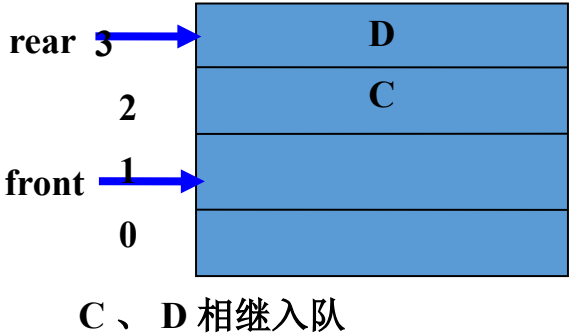
- 通常设队头指针指向队头元素前面一个位置，队尾指针指向队尾元素（这样的设置是为了某些运算的方便，并不是唯一的方法）。

顺序队列运算时的头、尾指针变化









顺序队列的主要运算（一）

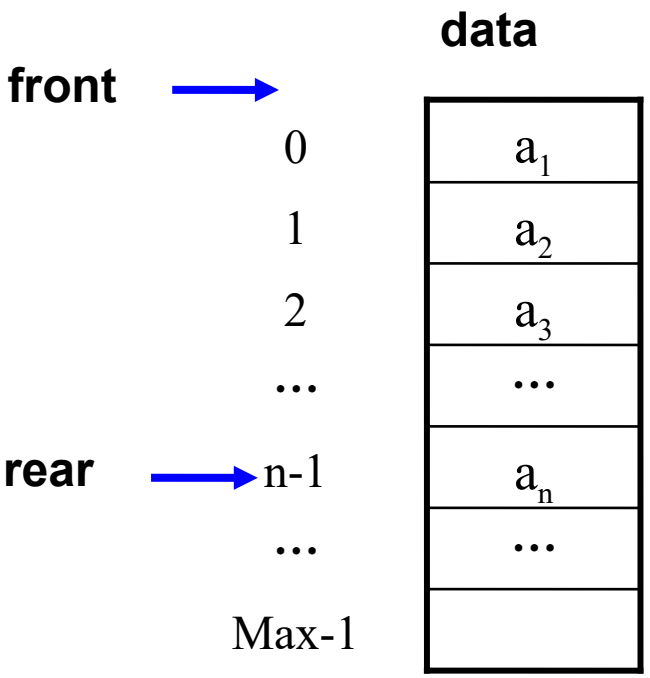
初始状态：

rear==front= -1

队列长度：

m=(sq->rear)-(sq->front);

队满时 m=maxsize，队空时 m=0



顺序队列的主要运算（二）

- 入队：

在不考虑溢出的情况下，入队操作队尾指针加 1，指向新位置后，元素入队。

操作如下：

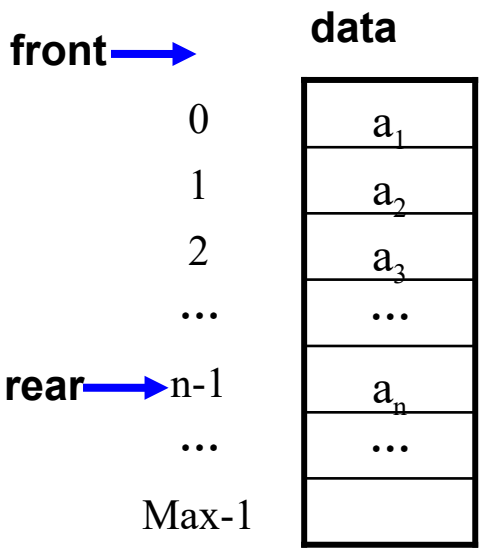
```
sq->rear++;
sq->data[sq->rear]=x;
```

- 出队：

在不考虑队空的情况下，出队操作队头指针加 1，表明队头元素出队。

操作如下：

```
sq->front++;
x=sq->data[sq->front];   /* 原队头元素送 x 中 */
```



- 设 MAXSIZE=10 。

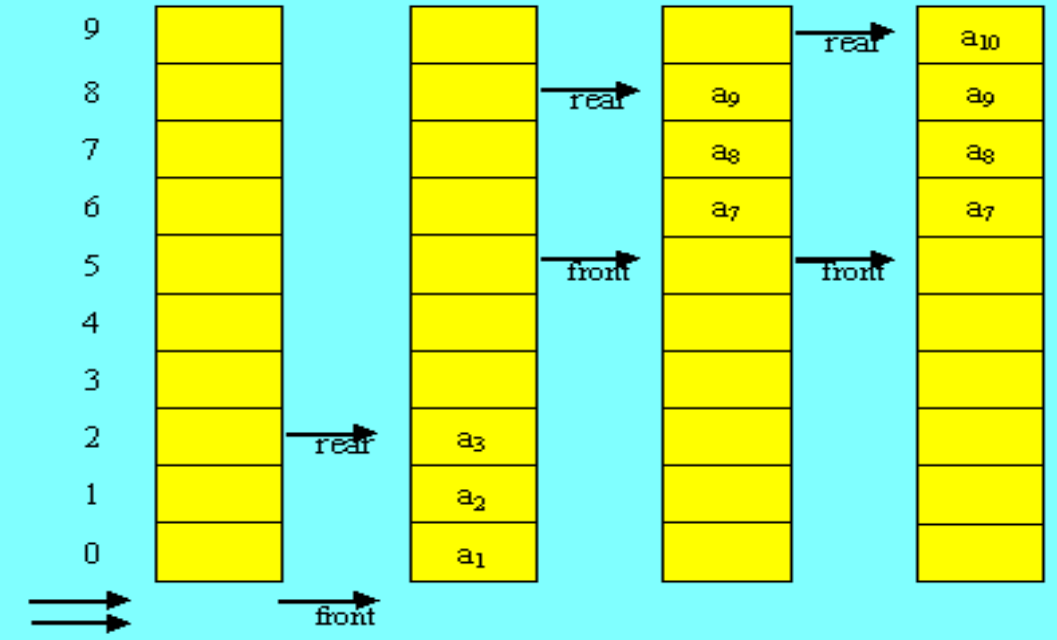


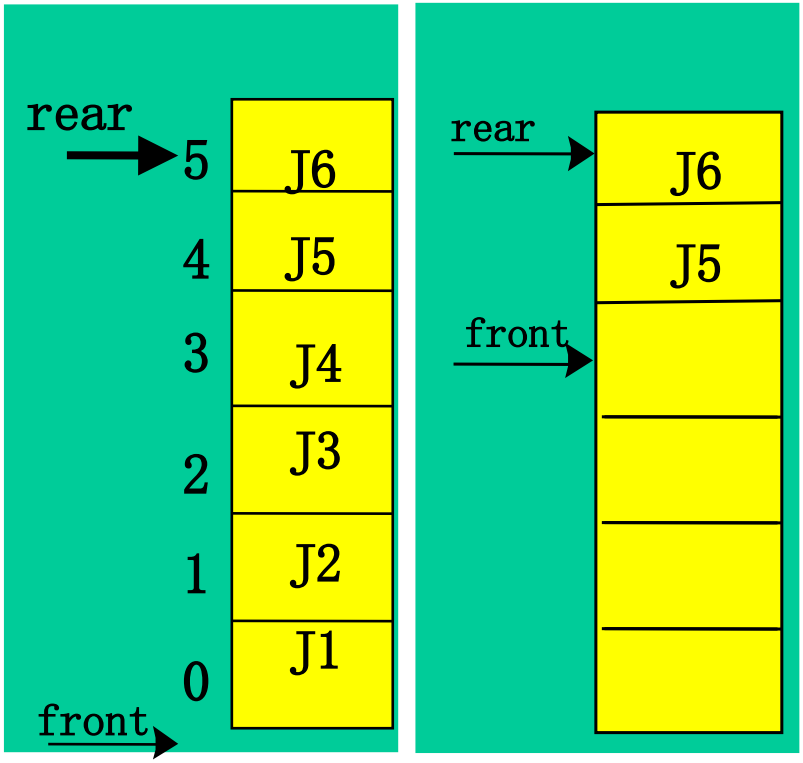
图 3-11 队列操作示意图

顺序队列的上溢

队溢出：

真溢出：队列真正满时再入队 (rear – front == maxsize) 。

假溢出： rear 已指向队尾，但队列前端仍有空位置。

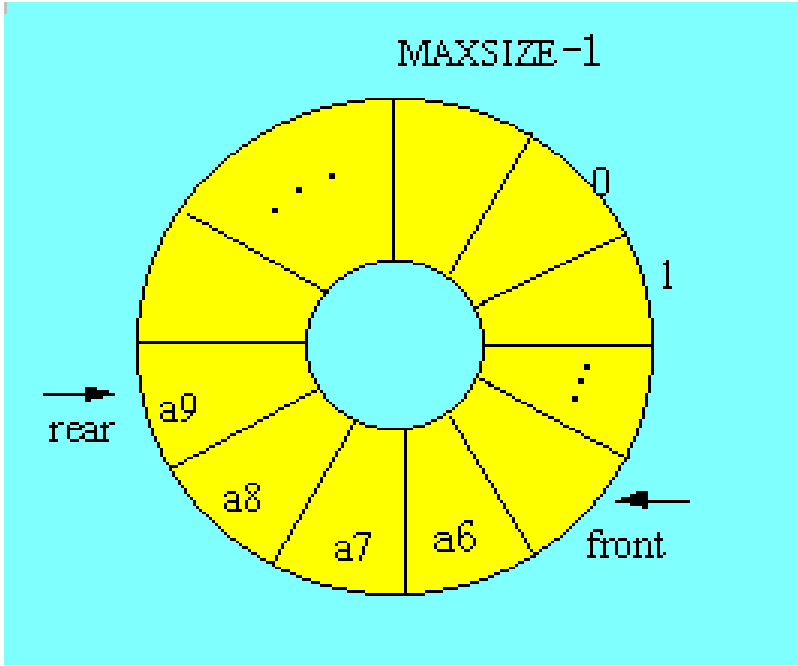


解决假溢出的方法：

方法一：每次删除队头一个元素后，把整个队列往前移一个位置（造成时间浪费）。

方法二：循环队列

循环队列



data[0] 接在 data[Maxsize-1] 之后

若 rear+1==Maxsize

则令 rear=0;

循环队列的操作实现

- 尾指针的加 1 操作可描述为：

```
If  (sq->rear==maxsize-1)
    sq->rear=0;
else
    sq->rear++;
```

利用“取余运算”，则运算可描述为：

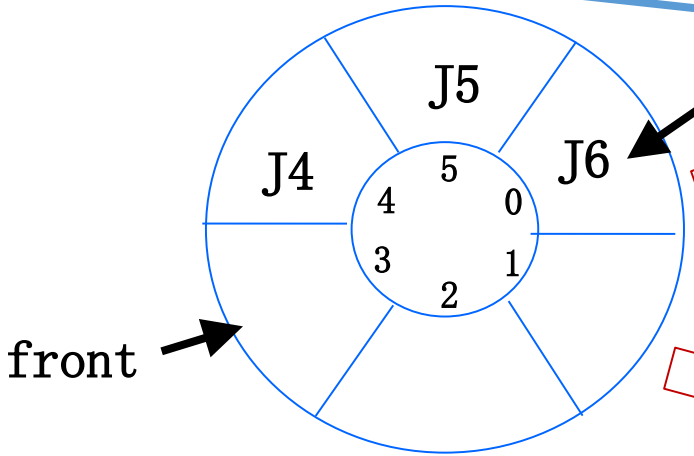
入队： sq-> rear = (sq-> rear+1)%maxsize

同样道理，

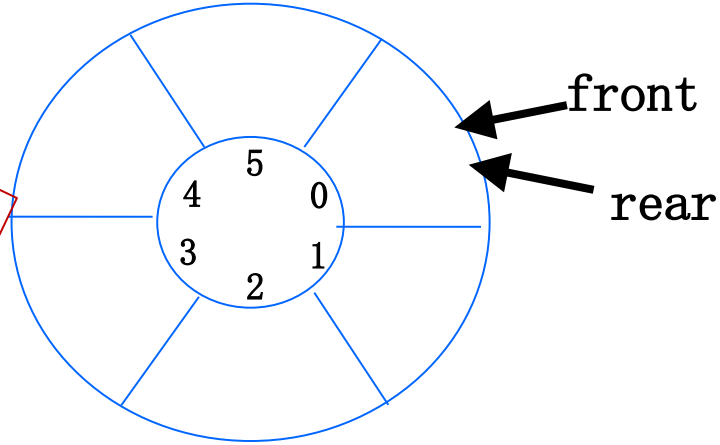
出队： sq-> front =(sq-> front+1)%maxsize

队空： front＝rear

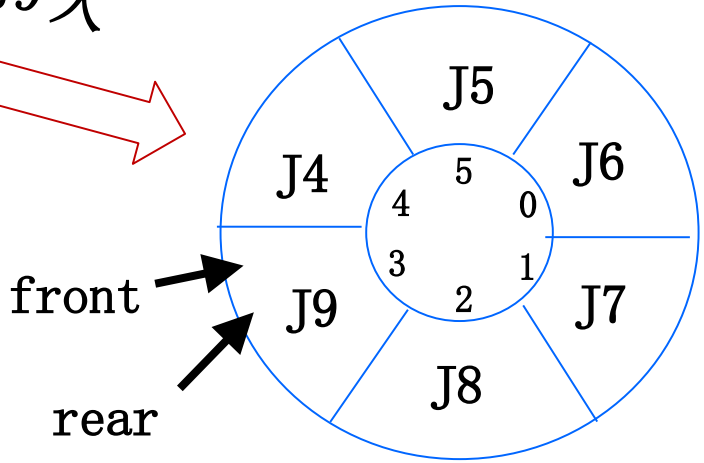
队满： front＝rear



J4, J5, J6 出队

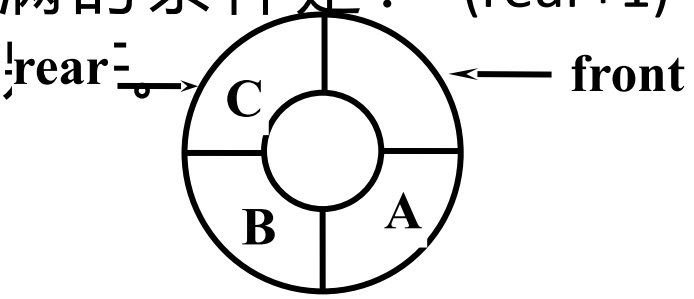


J7, J8, J9 入队



如何解决“队满”和“队空”的条件相同这个问题?

- （1）附设一个存储队中元素个数的变量如 num，当 num==0 时队空，当 num==MAXSIZE 时为队满。
- （2）少用一个元素空间，把留有一个元素的存储空间的情况视为队满，此时的状态是队尾指针加 1 就会从后面赶上队头指针，这种情况下队满的条件是：(rear+1) % MAXSIZE ==front，也能和空队区别



Size = 4

Count = 3

循环队列的类型定义如下：

```
typedef  struct    {
    datatype data[MAXSIZE];      /* 数据的存储区 */
    int front,rear;          /* 队头队尾指针 */
    int num;                 /* 队中元素的个数 */
  }c_SeQueue;            /* 循环队 */
```

循环队列的操作实现

- 尾指针的加 1 操作可描述为：

  If（rear+1>=maxsize)　rear=0; else rear++;

- 利用“取余运算”，则运算可描述为：

  队空：　r == f

  队满：　(r+1)%maxsize ==f

  入队：　r=(r+1)%maxsize;

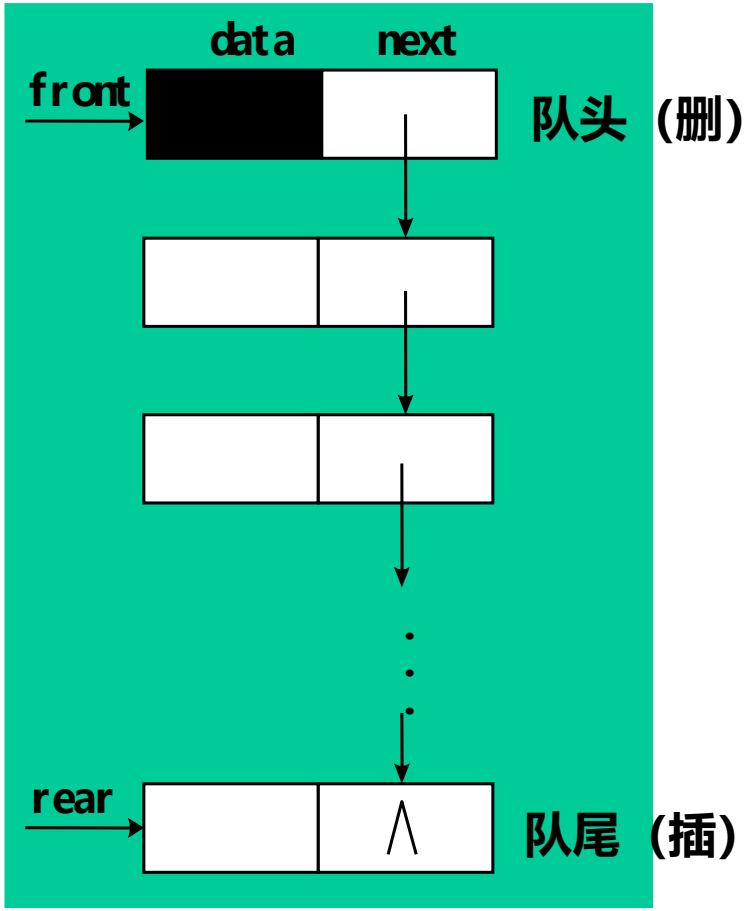
  出队：　f=(f+1)%maxsize;

- 循环队列的几种运算描述：

  置空队、判空队、入队、出队、取队头元素

链队列

- 队列的链式存储结构
- 用一个带头结点的单链表来表示队列。
- 把线性链表的头指针定义为队头指针 front，让其指向头结点；在链表最后结点建立一个队尾指针 rear；并且限定只能在链头进行删除，在链尾进行插入，这个单链表就构成了一个链队列。



链队列的几种基本运算

1. 置空队

   front=rear=malloc() ; front->next=NULL;

2. 判断队空

   front->next==NULL;

3. 取队头结点数据

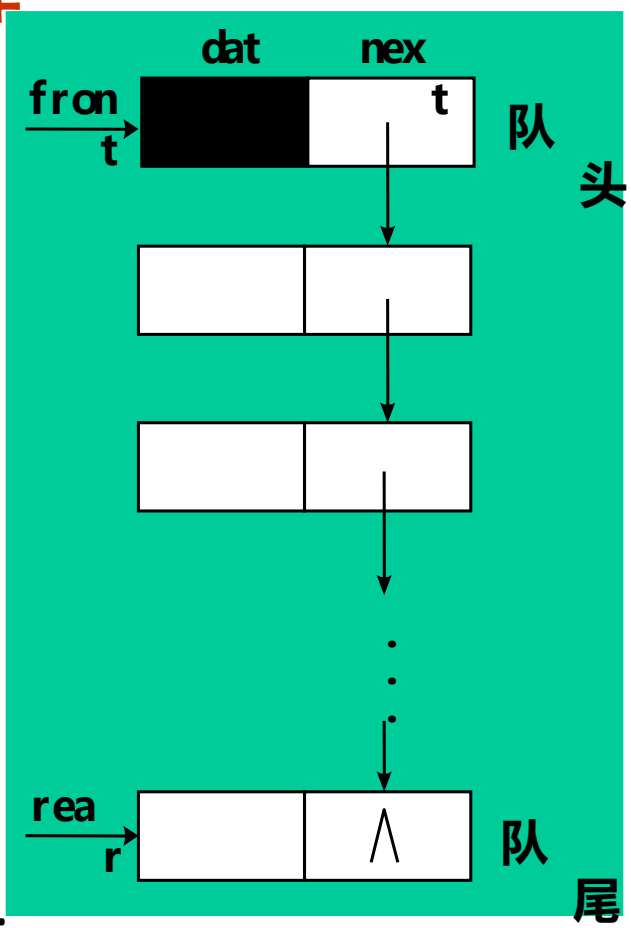
   x=front->next->data;

4. 入队

   p=malloc(); rear->next=p; rear=p;

5. 出队

   q=front->next; front->next=q->next; free(q);



复习

1. 队列和线性表有什么不同?
2. 实现循环队列的“判队空或满”的方法有哪些?
3. 在链队列中如何“判队空”和“判队满”？

队列限定插入只能在表的一端和删除只能在表的另一端进行

使用循环队列的最主要的技术问题是如何判断其“满”和“空”的状态。

1）少用一个元素的存储空间，头尾指针相同作为队空，“队列头指针在队列尾指针的下一位置上” 作为队列满的标志。

2）使用一个计数器记录队列中元素的总数

队空条件： front ==rear

链队列不需判队满